CS3230: Design and Analysis of Algorithms (Spring 2020)

Homework Set #1

OUT: 24-Jan-2020 **DUE:** Thursday, 6-Feb-2020, 6:00pm

IMPORTANT: Write your NAME, Matric No, Tut. Gp in your Answer Sheet.

- Start each of the problems on a *separate* page.
- Make sure your name and matric number is on each sheet.
- Write legibly. If we cannot read what you write, we cannot give points. In case you CANNOT write legibly, please type out your answers and print out hard copy.
- To hand the homework in, **staple them together** and drop them into the CS3230 dropbox in undergraduate general office before the due date and time.
- When you submit your homework, please try to make it short. The page limit is 8.

IMPORTANT:

You are advised to start on the homework *early*. For some problems, you need to let it incubate in your head before the solution will come to you. Also, start early so that there is time to consult the teaching staff during office hours. Some students might need some pointers regarding writing the proofs, others may need pointers on writing out the algorithm (idea, example, pseudo-code), while others will need to understand *more deeply* the material covered in class before they apply them to solving the homework problems. Use the office hours for these purposes!

It is always a good idea to email the teaching staff before going for office hours or if you need to schedule other timings to meet. Do it in advance.

HOW TO "Give an Algorithm":

When asked to "give an algorithm" to solve a problem, your write-up should *NOT* be just the code. (*This will receive very low marks*.) Instead, it should be a short essay. The first paragraph should summarize the problem and what your results are. The body of the essay should consist of the following:

- A description of the algorithm *in English* and, if helpful, pseudo-code.
- One *worked example or diagram* to show more precisely how your algorithm works.
- A *proof* (or indication) of the correctness of the algorithm
- An *analysis* of the running time of the algorithm.

Remember, your goal is to communicate. Full credit will be given only to correct solutions which are described clearly. Convoluted and obtuse descriptions will receive low marks.

In CS3230, you learn to develop high-level abstractions when describing algorithms. Try not to speak in ML/AL (machine/assembly language) or "for (j=0; j<n; j++) do". Instead give names to your sets (of objects or things or data structures), talk about Depth-First Search, Binary Search, traverse the graph, sort the set, use a priority queue, etc. You are no longer in CS1010, CS1020, CS2010 or CS2020. Speak with greater sophistication, and at a higher level of abstraction.

(**Note:** Each problem is worth 10 marks.)

S1. Modified from Problem 3-3, pp 61-62 of [CLRS] [Sorting out order of growth rates]

Rank the following functions in *increasing order of growth*; that is, if function $f(n)$ is *immediately* before function $g(n)$ in your list, then it should be the case that $f(n)$ is $O(g(n))$.

$g_1(n) = n!$	$g_2(n) = 3n(\lg n)$	$g_3(n) = n^{\lg n}$	$g_4(n) = lg\ (n!)$
$g_5(n) = n^{(1/\lg n)}$	$g_6(n) = (\lg n)!$	$g_7(n) = 2^n - n^2$	$g_8(n) = n(\lg \lg n)$

To simplify notations, we write $f(n) << g(n)$ to mean $f(n) = o(g(n))$ and $f(n) \equiv g(n)$ to mean $f(n) = \Theta(g(n))$. For example, the four functions n^2, n, $(2013n^2 + n)$ and n^3 could be sorted in increasing order of growth as follows: ($n << n^2 \equiv (2013n^2 + n) << n^3$).

Do not turn in proofs for this problem, but you should do them anyway just for practice.
(*Note:* $\lg n = \log_2 n$.)

S2. [Copy from Term Test 2018/2019 Sem2, Solving recurrence]

(i) $T(n) = 2\ T(n/2) + n \lg \lg n$
(ii) $T(n) = 4\ T(n/2) + n$
(iii) $T(n) = 4\ T(n/2) + n \lg \lg n$
(iv) $T(n) = 4\ T(n/2) + n^2$
(v) $T(n) = 4\ T(n/2) + n^2 \lg \lg n$
(vi) $T(n) = 4\ T(n/2) + n^3$
(vii) $T(n) = 4\ T(n/2) + n^3 \lg \lg n$

(a) Among the above recurrences, please indicates those that can be solved by master theorem. For each recurrence that is solvable by master theorem, please indicate which case it belongs to (either case 1, 2, or 3) and state the time complexity.
(b) For recurrences that are not solvable by master theorem, please solve them.

S3. [Iterative algorithm]

Consider the function fun(x, y). What is its output? What is the invariant for the while loop? Can you show that this algorithm correctly computes the output?

```
fun(x, y)
ans=0; p=x; q=y;
while (q>0) do
    r = q mod 2; q = q div 2;
    ans = ans + r*p;
    p = p * 2;
return ans;
```